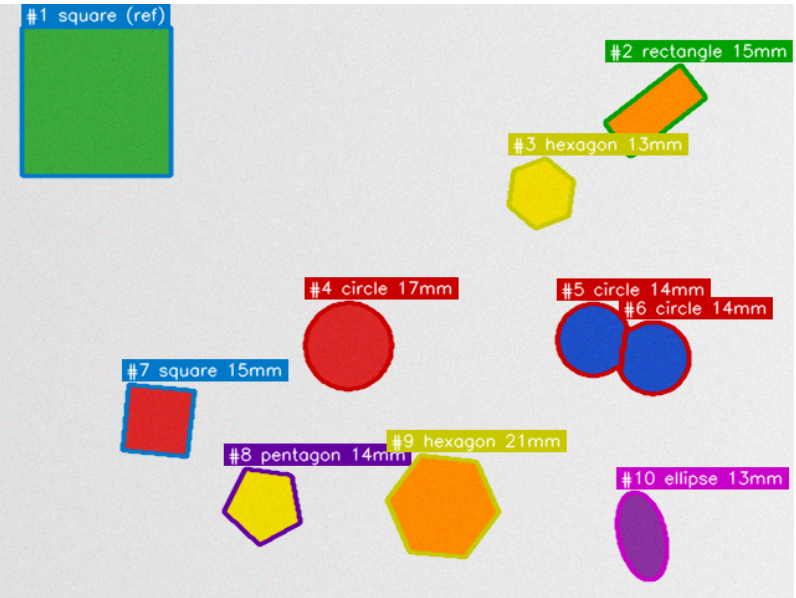
VIT Bhopal University

B.Tech Computer Science and Engineering (AI & ML)

VITyarthi - Build Your Own Project

ObjectLens

Automated Object Detection, Classification and Real-World Measurement using Computer Vision



Course	Computer Vision
Project type	Build Your Own Project (individual)
Submitted by	**Ashish Kumar Rai**
Registration number	**24BAI10666**
GitHub repository	https://github.com/ashishtech-03/ObjectLens
Month / Year	October 2026

Table of Contents

Table of Contents 2

1. Introduction 3

2. Problem Statement 3

3. Functional Requirements 3

4. Non-functional Requirements 4

5. System Architecture 6

6. Design Diagrams 7

6.1 Use Case Diagram . . . 7

6.2 Workflow Diagram . . . 7

6.3 Sequence Diagram . . . 8

6.4 Class / Component Diagram . . . 9

6.5 ER Diagram and Schema Design . . . 10

7. Design Decisions and Rationale 12

8. Implementation Details 13

8.1 Pre-processing . . . 13

8.2 Segmentation . . . 13

8.3 Feature extraction . . . 13

8.4 Classification, training data and model . . . 14

8.5 Calibration and measurement . . . 14

8.6 Reporting, storage and CLI . . . 14

8.7 Version control . . . 14

9. Screenshots and Results 15

9.1 Pipeline stages . . . 15

9.2 Output on different backgrounds . . . 15

9.3 Classifier evaluation . . . 16

9.4 End-to-end evaluation . . . 16

9.5 Robustness to image noise . . . 17

10. Testing Approach 18

11. Challenges Faced 18

12. Learnings and Key Takeaways 19

13. Future Enhancements 19

14. References 20

1. Introduction

Counting, sorting and sizing small objects - machine parts, laboratory samples, tokens, seeds - is a routine task in workshops, laboratories and classrooms. Done by hand it is slow, tiring and error-prone; commercial machine-vision systems solve it but are expensive, need special hardware and are black boxes. **Computer vision** offers a middle path: a single ordinary photograph already contains the information needed to find every object, recognise it and measure it.

ObjectLens is an end-to-end computer-vision application built for the *Computer Vision* course. Given a photograph of objects on a plain background it automatically (1) detects and separates every object, including objects that touch each other, (2) classifies each object's shape and colour with a trained machine-learning model, (3) converts pixel sizes to millimetres using a reference marker and (4) produces an annotated image, CSV/JSON reports, a summary chart and a searchable history database.

The project applies the central topics of the syllabus in one coherent system: image representation and colour spaces (BGR, grey, HSV, CIE-Lab), filtering and denoising, contrast enhancement (CLAHE), thresholding (Otsu), mathematical morphology, distance transform and watershed segmentation, contour analysis, shape descriptors and image moments (Hu invariants), supervised classification, camera-scale calibration and quantitative evaluation. All results in this report are reproducible with the commands in the repository README.

Repository: https://github.com/ashishtech-03/ObjectLens

2. Problem Statement

Design and implement a low-cost, transparent software system that, from a single photograph of objects lying on a plain background, can **detect** every object (even when objects touch), **classify** its shape and colour, **measure** its real-world dimensions in millimetres, and **report** the results in a form that can be inspected, exported and stored, while quantifying its own accuracy and robustness.

Objectives

- Detect all objects, including touching ones, with high precision and recall.
- Classify eight shape classes and name the object colour, flagging uncertain predictions as *unknown*.
- Calibrate pixels to millimetres from a reference square and report length, width, area and perimeter.
- Provide reproducible outputs (annotated image, CSV, JSON, chart) and a history database.
- Evaluate rigorously on data with exact ground truth, including a noise-robustness study.
- Deliver clean, modular, tested and documented code under version control.

Scope. In scope: classical CV pipeline, ML shape classifier, calibration, synthetic data generator, CLI, SQLite, tests. Out of scope: deep-learning detectors, perspective correction, live video, cluttered or textured backgrounds. **Target users:** students and educators, small workshops and makers, developers needing a readable reference pipeline.

3. Functional Requirements

The system provides five major functional modules, each with a clear input/output contract:

Module	Input	Processing	Output
M1 Detection & segmentation	Image file	Validation, resize, Gaussian blur, CLAHE, Lab background distance, Otsu, morphology, watershed, edge refinement	Per-object contours and masks

Module	Input	Processing	Output
M2 Recognition	Contours + image	Shape, moment and colour features; SVM classification with confidence	Label, confidence, colour name
M3 Measurement & calibration	Features + reference/scale	px/mm from reference square; unit conversion	Length, width, area, perimeter in mm
M4 Reporting & storage	Analysis result	Annotation, CSV/JSON/chart export, SQLite transaction	Files + database rows + console summary
M5 Training & evaluation	Seeds, sample counts	Synthetic scene generation, training, CV, end-to-end metrics, noise sweep	Model file, metrics.json

ID	Functional requirement
FR1	Load and validate an image (existence, extension whitelist, decodability).
FR2	Pre-process: resize large images, Gaussian denoising, CLAHE contrast enhancement.
FR3	Segment foreground on light or dark plain backgrounds, independent of object colour.
FR4	Separate touching objects (distance transform + watershed).
FR5	Extract area, perimeter, circularity, aspect ratio, extent, solidity, vertex counts, Hu moments and colour.
FR6	Train, save, load and apply an ML classifier; output confidence; label low-confidence objects *unknown*.
FR7	Calibrate pixels-per-millimetre from a reference square or a user-supplied value.
FR8	Measure length, width, area, perimeter and equivalent diameter in millimetres.
FR9	Export annotated image, binary mask, CSV, JSON and a per-shape count chart.
FR10	Store every analysis in SQLite; list stored runs and inspect their detections.
FR11	Generate synthetic annotated scenes; evaluate the classifier and the whole pipeline; compare models.
FR12	Provide a command-line interface: train, analyze, demo, evaluate, history.

Logical workflow of user interaction. The user first trains the model once (`python main.py train`), then analyses an image with optional calibration flags, inspects the annotated image and exported files, optionally stores the run, and later reviews history. The complete workflow diagram is given in Section 6.

4. Non-functional Requirements

ID	Category	Requirement	Realisation / verification
NFR1	Performance	< 0.2 s for 640x480 and < 0.5 s for 1920x1440 images on a laptop CPU.	Images are down-scaled to at most 1024 px; vectorised OpenCV/NumPy. Measured about 49 ms and 144 ms.
NFR2	Reliability and error handling	Invalid input must never crash the program with a stack trace.	Custom exception hierarchy; the CLI returns exit code 2 with a clear message for expected errors and code 1 plus a logged traceback for unexpected ones. Covered by tests.
NFR3	Security	No SQL injection; all inputs validated.	Parameterised queries only, integer coercion of ids, extension whitelist, configuration validation. Models are loaded only from user-trained files (documented).
NFR4	Usability	Self-explanatory interface and readable results.	Help text for every command, annotated images, plain-text summary, actionable error messages (e.g. 'Run: python main.py train').
NFR5	Maintainability	Small single-purpose modules, documented and tested.	15 modules, 1553 lines in the package, type hints, docstrings, 70 automated tests.
NFR6	Logging / monitoring	Every run leaves an audit trail.	Console and rotating file log (outputs/objectlens.log) with configurable level.

ID	Category	Requirement	Realisation / verification
NFR7	Resource efficiency	CPU only, modest memory.	Classical CV plus a small SVM (model file under 1 MB); no GPU or large downloads.
NFR8	Reproducibility	Same seed gives same results.	Seeded NumPy generators and scikit-learn random_state.

5. System Architecture

ObjectLens follows a layered architecture. The **presentation layer** (CLI) parses commands and handles errors. The **application layer** contains the pipeline orchestrator, the validated immutable configuration and the evaluation code. The **computer-vision core** is a linear chain of five stages, each in its own module with a narrow interface (pre-processing, segmentation, feature extraction, classification, measurement). The **output and persistence layer** holds reporting, the SQLite repository and the stored model. A synthetic data generator supplies labelled data for training and evaluation only.

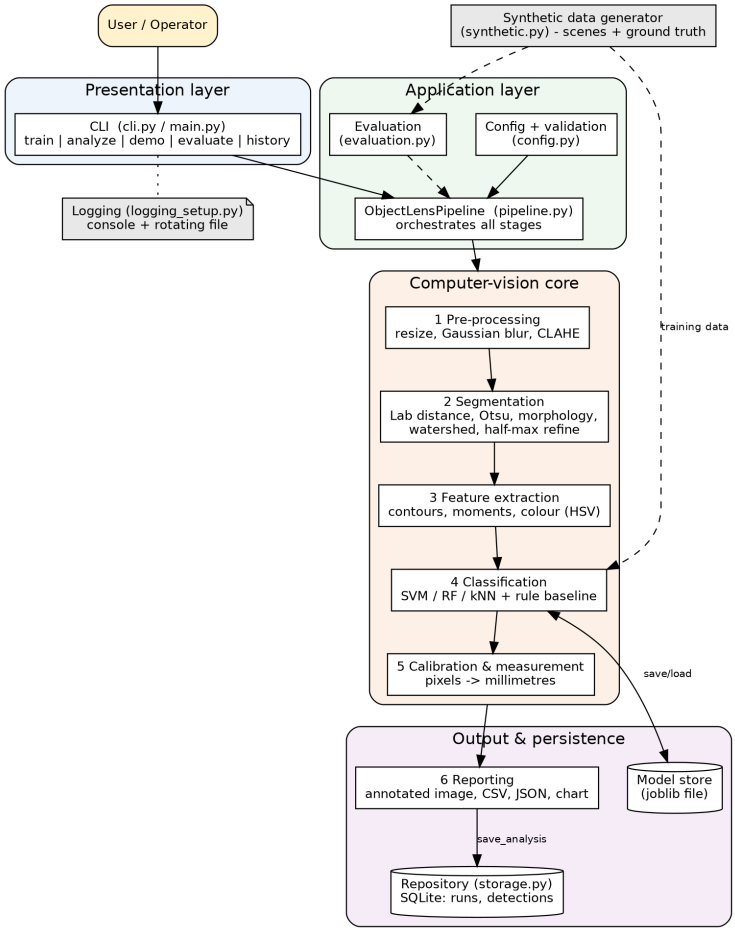


Figure 1 - System architecture of ObjectLens

This separation lets each stage be tested in isolation (for example segmentation is tested with hand-drawn images, the classifier with generated feature vectors) and replaced independently (for example swapping the SVM for a neural network would only touch `classifier.py`).

6. Design Diagrams

6.1 Use Case Diagram

Two actors use the system: the **Operator** analyses images, calibrates, exports and reviews history, while the **Developer / Evaluator** trains and evaluates models. Analysing an image *includes* detection, classification and measurement; measurement includes calibration; storing the result is an optional *extension*.

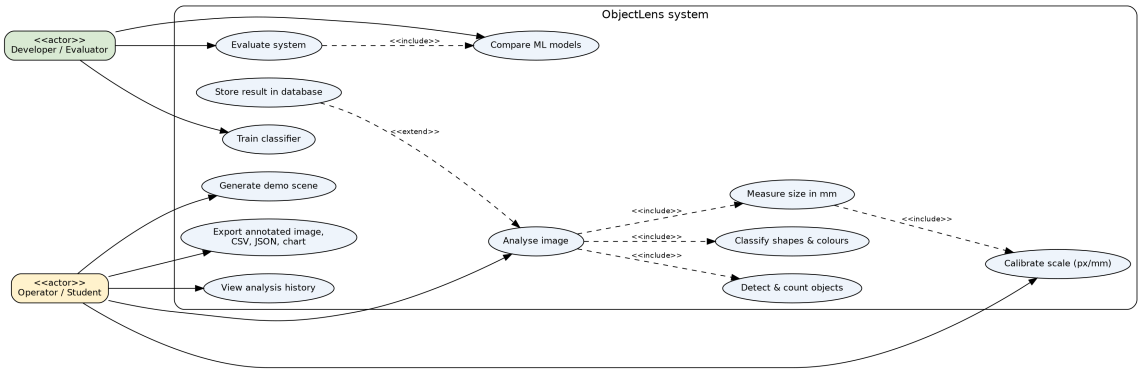


Figure 2 - Use case diagram

6.2 Workflow Diagram

The workflow shows the decisions taken while analysing an image: file validation, optional watershed splitting, optional calibration and optional database storage.

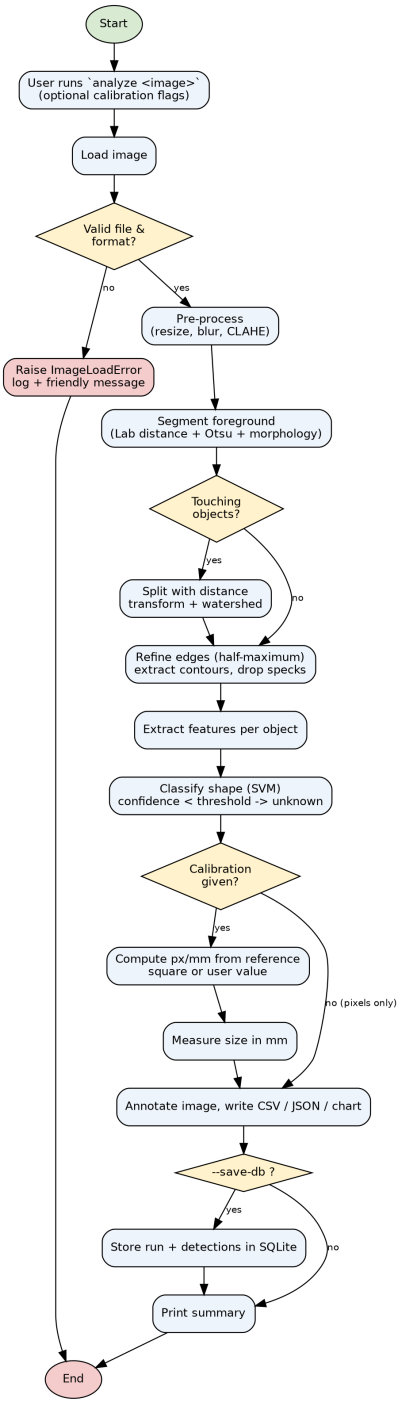


Figure 3 - Process / workflow diagram of the analyse command

6.3 Sequence Diagram

The sequence diagram shows the interaction between user, CLI, pipeline and the stage modules for one analysis, including the report-writing and optional persistence calls.

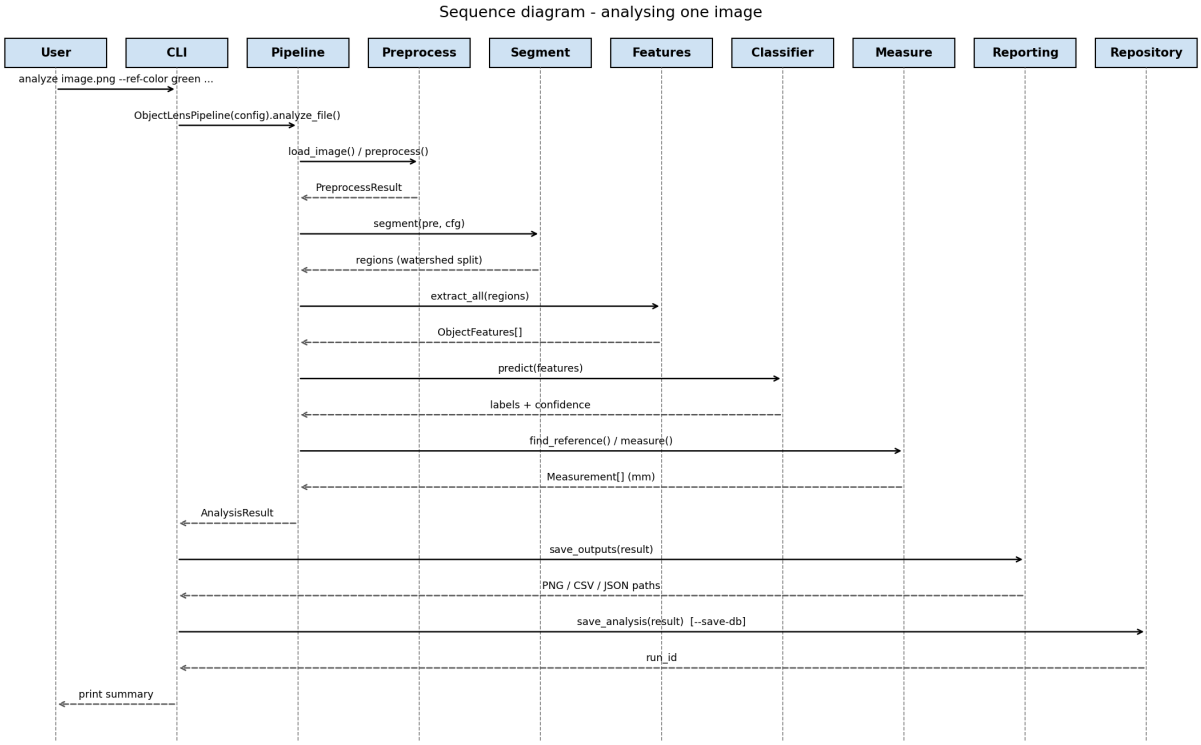


Figure 4 - Sequence diagram for analysing one image

6.4 Class / Component Diagram

`ObjectLensPipeline` composes a `Config` and a `ShapeClassifier` and produces an `AnalysisResult` holding one `DetectedObject` per object, each of which aggregates a `Region`, `ObjectFeatures` and an optional `Measurement`. `Repository` persists results; all expected failures derive from `ObjectLensError`.

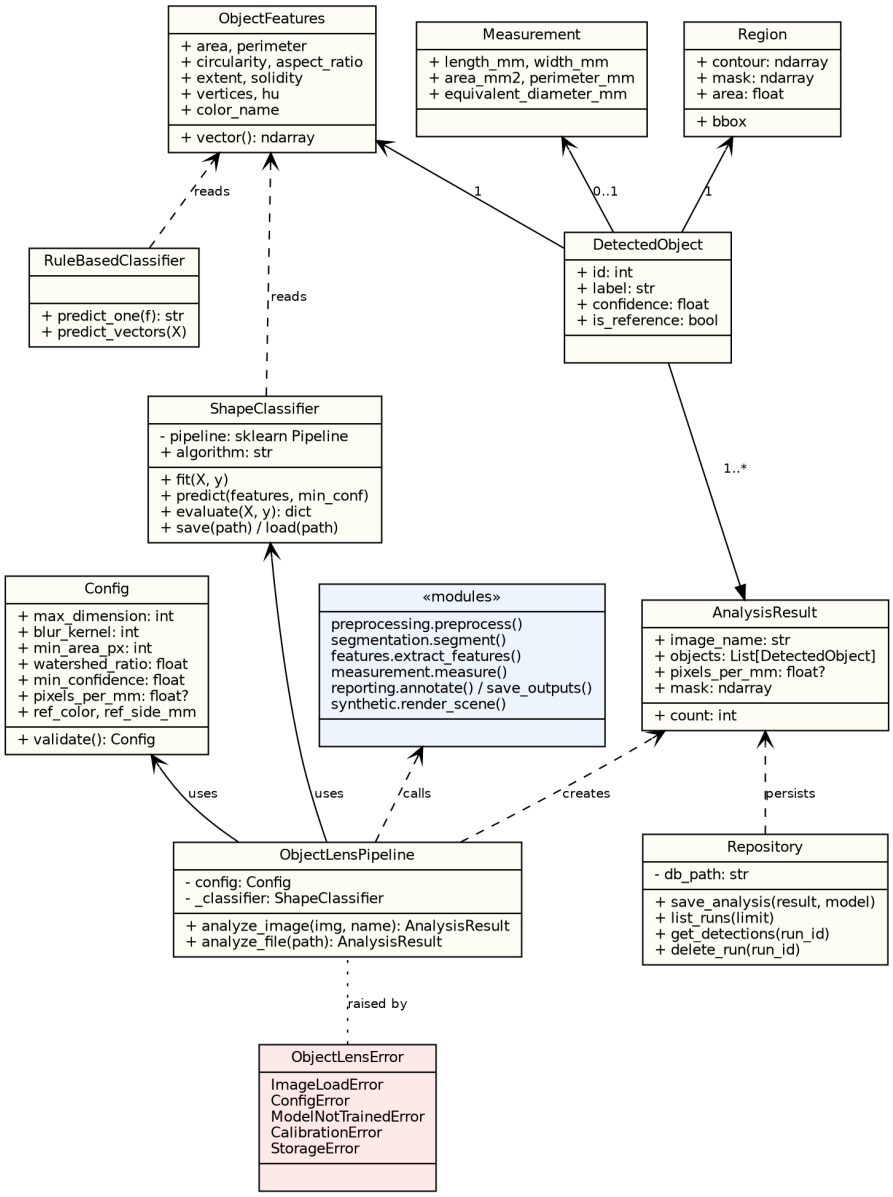


Figure 5 - Class / component diagram

6.5 ER Diagram and Schema Design

Every analysis is stored as one row in `runs` and one row per object in `detections` (one-to-many, cascade delete). The two inserts happen in a single transaction so a run is never stored half-way. An index on `detections.run_id` keeps look-ups fast.

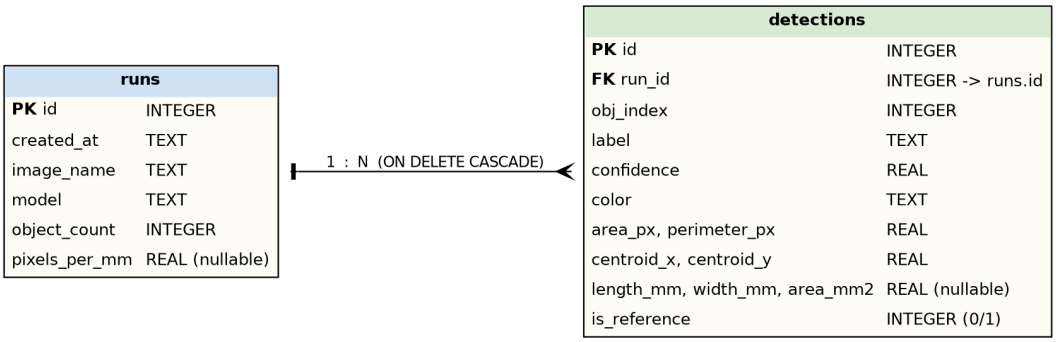


Figure 6 - Entity-relationship diagram of the SQLite database

7. Design Decisions and Rationale

Decision	Alternatives considered	Rationale
Segment with **Lab distance from the border colour**	Grey-level Otsu; adaptive threshold; Canny edges	Independent of object colour and brightness; handles light and dark backgrounds without a polarity hack; yellow-on-white objects are not lost. Canny edges give open contours.
Clamped Otsu threshold (15-40 Lab units)	Pure Otsu; fixed threshold	Pure Otsu fails with a very high-contrast object next to a low-contrast one, or on an empty image; a fixed value is not adaptive.
Half-maximum edge refinement	Use the first mask as is	The permissive global threshold over-grew masks (area error 4-7 %). Moving each object's boundary to 50 % of its own contrast removed the bias (median error 0.64 %).
Distance transform + watershed with solidity gate, adaptive ratio and noise-core filter	Hough circles; erosion-based splitting	Works for any convex-ish object, not only circles; the solidity gate guarantees convex objects are never split; filtering tiny cores prevents sliver objects.
Scale-invariant 8-D feature vector	Raw pixels + CNN; HOG	Small, interpretable, trains in seconds on CPU, and independent of object size and rotation.
SVM (RBF) as the production model	Random Forest, k-NN, rules	Smooth class probabilities for confidence thresholding; strong on small data. Alternatives were benchmarked (Section 9).
Train on features of the **real pipeline output**	Compute features from ideal geometry	Removes train/inference mismatch (digital contour perimeter bias, noise).
Reference-square calibration	Camera intrinsics; checkerboard	One visible marker gives px/mm per image; no calibration session needed.
Confidence threshold -> unknown	Always output the arg-max	Flags uncertain objects instead of reporting wrong shapes confidently.
SQLite repository	CSV only; client/server DB	Zero-configuration, transactional, cascade delete, built into Python.
Synthetic generator with exact ground truth	Manual annotation of photos	Reproducible, unlimited, and gives exact area/position truth so size error can be measured automatically.

8. Implementation Details

The package `objectlens` contains the modules below (lines of code in brackets).

Module	LOC	Responsibility
`__init__.py`	4	package metadata
`classifier.py`	149	ShapeClassifier (SVM/RF/kNN), RuleBasedClassifier, model comparison
`cli.py`	177	argparse CLI and error handling
`config.py`	59	immutable Config with validation
`evaluation.py`	106	evaluation of classifier, end-to-end pipeline and robustness
`exceptions.py`	25	custom exception hierarchy
`features.py`	124	shape / moment / colour feature extraction
`logging_setup.py`	30	console + rotating-file logging
`measurement.py`	58	px/mm calibration and mm measurements
`pipeline.py`	91	orchestration of all stages
`preprocessing.py`	94	image loading, validation, resize, blur, CLAHE
`reporting.py`	141	annotation, CSV, JSON, chart export
`segmentation.py`	185	Lab-distance segmentation, morphology, watershed, edge refinement
`storage.py`	113	SQLite repository
`synthetic.py`	197	synthetic scene and dataset generator

8.1 Pre-processing

Images are validated, converted to 3-channel BGR, down-scaled with area interpolation when larger than 1024 px (the scale factor is remembered so user-supplied calibration still applies), smoothed with a 5x5 Gaussian kernel and a grey version is enhanced with CLAHE (clip limit 2, 8x8 tiles) for contrast-limited local equalisation.

8.2 Segmentation

The background colour is the per-channel median of the border pixels in CIE-Lab, where Euclidean distance approximates perceptual colour difference. For every pixel *p* the distance map is d(p) = || Lab(p) - Lab_{bg} ||. The map is thresholded with Otsu's method, clamped to 15-40 Lab units, and cleaned by morphological opening (removes speckle) then closing (fills pin-holes). Connected components that are not convex (solidity below 0.96) are analysed with the distance transform; the threshold on the transform is raised from 55 % up to 85 % of its maximum until at least two significant cores (each at least 15 % of the largest) appear. These cores seed a marker-controlled watershed that splits the blob. Finally each object's mask is shrunk to its **half-maximum** edge: T_i = 0.5 x median(d inside the eroded object i). Objects smaller than 150 px are discarded.

8.3 Feature extraction

For every contour the system computes area A, perimeter P, circularity 4*pi*A/P^2, aspect ratio and extent from the minimum-area rotated rectangle, solidity (A divided by convex-hull area), the number of vertices of the polygon approximation (epsilon = 3 % of the perimeter) of the contour and of its hull, log-scaled Hu moments (first two used), and the mean colour of the eroded interior, mapped to a colour name through HSV ranges. The eight-element classifier vector is scale and rotation invariant.

8.4 Classification, training data and model

Training data are rendered by `synthetic.py` (8 classes x 150 samples + 50 watershed-split circles = 1300 samples; random size, rotation, colour, background theme, brightness gradient and noise) and are processed by the same segmentation and feature code as real images. A scikit-learn pipeline *StandardScaler -> SVC(RBF, C=10, probability=True)* is trained, saved with joblib and loaded on demand. Predictions with a probability below 0.45 are labelled *unknown*.

8.5 Calibration and measurement

If a reference colour and side length are given, the largest *square* of that colour is the reference; its side in pixels is the mean of sqrt(area) and the mean of its rectangle sides, and k = $side_{px}$ / $side_{mm}$. Each object then yields length = long side / k, width = short side / k, area = A / k^2, perimeter = P / k and equivalent diameter = 2*sqrt(A/pi) / k. Alternatively the user supplies pixels-per-mm directly; if the image was down-scaled the value is scaled accordingly.

8.6 Reporting, storage and CLI

`reporting.py` draws contours and labels, writes CSV and JSON (summary + per-object rows) and a bar chart. `storage.py` writes a run and its detections in one transaction. `cli.py` exposes five sub-commands, configures logging and converts exceptions to exit codes. Example session:

```
$ python main.py train
Trained svm on 1300 samples -> models/shape_classifier.joblib
$ python main.py analyze data/samples/scene_light.png --ref-color green --ref-side-mm 30 --save-db
Image          : scene_light.png
Objects found  : 9
By shape       : circle=3, ellipse=1, hexagon=2, pentagon=1, rectangle=1, square=1
Calibration    : 3.9999 px/mm
Saved to database as run #1
```

8.7 Version control

The project is developed in Git with small, descriptive commits (project skeleton, pre-processing and segmentation, features and classifier, measurement and storage, CLI, tests, documentation, report) so that the history documents the engineering process.

9. Screenshots and Results

9.1 Pipeline stages

Figure 7 shows the successive representations for one scene that contains a 30 mm green reference square and two touching circles: the input, the CLAHE grey image, the Lab distance map, the cleaned binary mask and the final classified and measured result. Labels show the object number, shape and equivalent diameter (*d*) in mm.

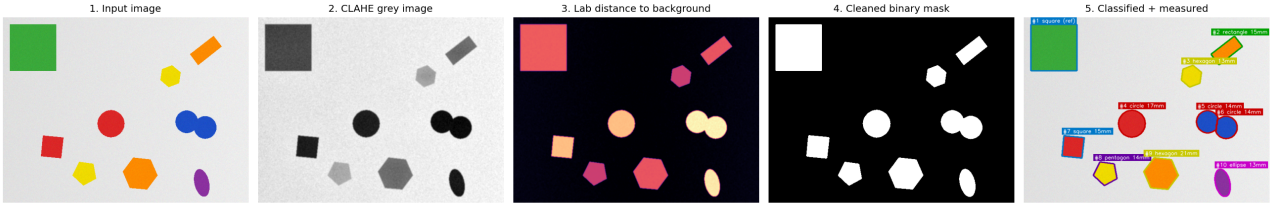


Figure 7 - Stages of the pipeline on one scene

9.2 Output on different backgrounds

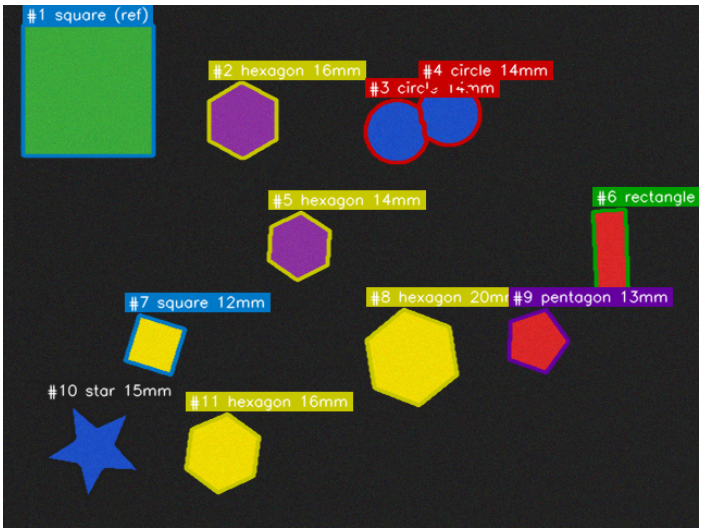


Figure 8 - Dark background

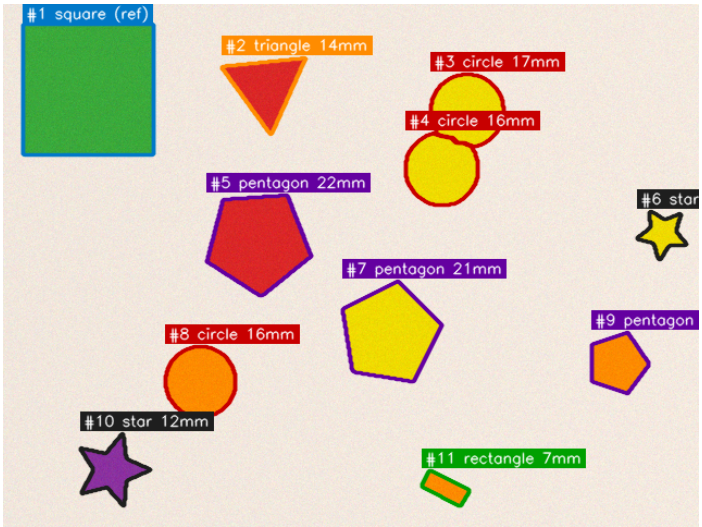


Figure 9 - Cream background

9.3 Classifier evaluation

The classifier was trained on 1300 samples and tested on 520 independent samples (65 per class): held-out accuracy = **100.0 %**. Table 4 compares the models by 5-fold stratified cross-validation on the training set.

Model	CV accuracy (mean)	Std. deviation
svm	99.92 %	0.15 %
random forest	100.00 %	0.00 %
knn	99.92 %	0.15 %
rule based	98.00 %	0.45 %

Table 4 - Cross-validated accuracy of the candidate models

The three ML models are essentially perfect on this data; the rule-based baseline reaches 98.0 % - it fails on shapes whose polygon approximation is ambiguous (for example circles and hexagons or elongated ellipses and rectangles). The SVM was kept for its calibrated probabilities.

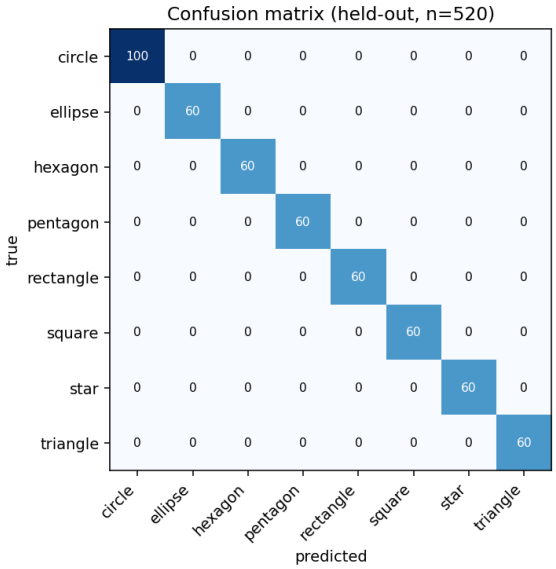


Figure 10 - Confusion matrix on the held-out test set

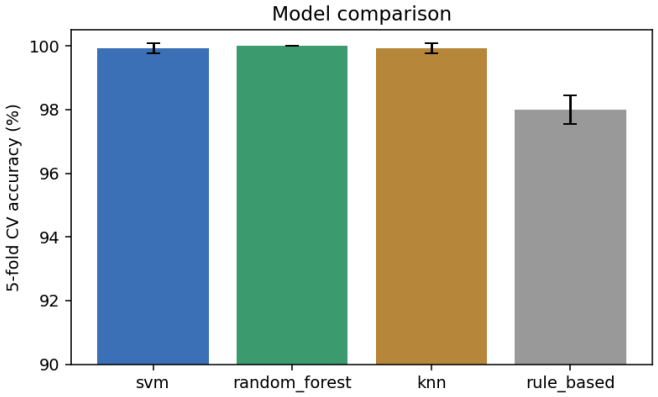


Figure 11 - Model comparison (5-fold CV)

9.4 End-to-end evaluation

40 freshly rendered scenes (348 objects, 14 scenes contain a touching pair) with exact ground truth were analysed after calibration from the reference square:

Metric	Result
Detection precision / recall	100.0 % / 100.0 %
Shape classification accuracy (matched objects)	100.0 %
Colour naming accuracy	100.0 %
Scenes with exactly the right object count	100.0 %
Area error (mean / median) after px/mm calibration	0.97 % / 0.64 %
Recovered calibration for a true 4.000 px/mm	3.9999 px/mm (mean over test scenes)
Run time per image (640x480 / 1920x1440)	about 49 ms / 144 ms

Table 5 - End-to-end results

9.5 Robustness to image noise

Gaussian noise sigma (grey levels)	Recall	Precision	Classification accuracy
5	100.0 %	100.0 %	100.0 %
15	100.0 %	100.0 %	100.0 %
30	100.0 %	100.0 %	100.0 %
45	100.0 %	100.0 %	100.0 %

Table 6 - Effect of sensor noise (20 scenes per level)

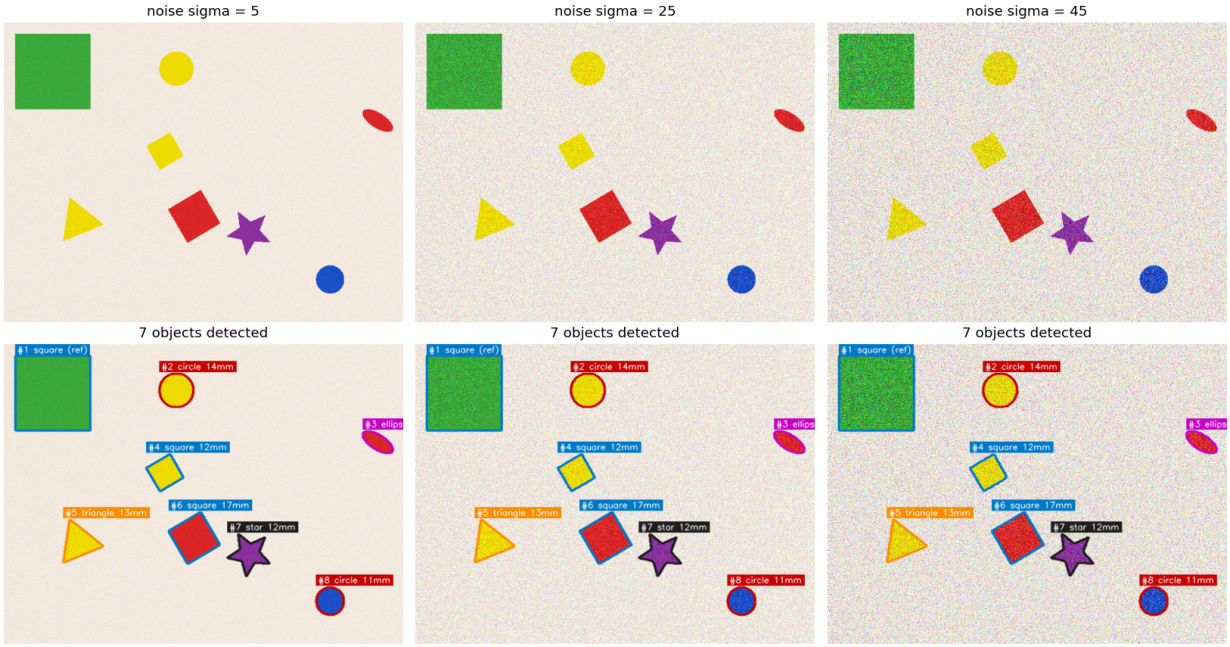


Figure 12 - Same scene at noise sigma 5, 25 and 45

Interpretation. Gaussian smoothing, Lab-distance segmentation and morphological opening absorb even strong noise, so performance does not degrade up to sigma = 45. This is a statement about *noise* only: the synthetic scenes have clean, evenly lit geometry, so these figures are an upper bound and real photographs (shadows, glare, perspective) will score lower. This limitation is discussed again in Sections 12 and 13.

10. Testing Approach

Testing uses **pytest** with 70 automated tests (all passing; about 10 s). The strategy has four layers:

- **Unit tests** for each module with hand-made images (for example a circle drawn with OpenCV must produce one region whose area is within 5 % of pi*r^2).
- **Validation / negative tests** for every error path: missing, unsupported and corrupt files, invalid configuration, untrained model, missing reference marker, unwritable database path.
- **Integration tests** that run the complete pipeline on generated scenes with ground truth, including a regression test (3 seeds) for the sliver-object bug found during development.
- **System / CLI tests** that call `main()` exactly as a user would (train, analyze with --save-db, history) and check exit codes and output.

Test file	Tests	What it covers
`test_classifier.py`	9	untrained / missing model errors, empty data, bad algorithm, held-out accuracy, save-load round trip, unknown label, baseline, model comparison
`test_config_preprocessing.py`	16	Config validation (8 invalid cases), missing / unsupported / corrupt files, colour-space and resize helpers
`test_features.py`	11	circle and rectangle feature values, vector shape, 8 colour-name cases
`test_measurement.py`	8	scale from reference, invalid sizes, mm conversion, reference selection and errors
`test_pipeline_cli.py`	12	full pipeline with touching pairs, regression test for sliver objects, resize + calibration, missing reference, blank image, end-to-end metrics, CLI train/analyze/history, graceful CLI errors
`test_segmentation.py`	7	blank image, dark background, specks, single circle area, touching circles split / merged, separate objects
`test_storage_reporting.py`	7	SQLite round trip, cascade delete, unwritable path, SQL-injection safety, annotation, summary, exported files

Table 7 - Test suite overview

Run with `python -m pytest -q`. Beyond the unit tests, the quantitative evaluation of Section 9 is itself a validation test of the whole system against ground truth, and is repeated with `python main.py evaluate`.

11. Challenges Faced

Challenge	Symptom	Resolution
Touching objects merged into one blob	Two touching circles were reported as one 'ellipse'; recall 92.5 %.	The solidity gate (0.92) was too strict for a mild overlap; it was raised to 0.96 and the watershed threshold became adaptive (55 % to 85 %). Recall rose to 100 %.
Spurious 'sliver' object	An extra 4 mm 'unknown' object appeared between two touching circles.	A 3-pixel noise core was used as a watershed seed. Cores smaller than 15 % of the largest are now ignored and a regression test covers it.
Systematic area over-estimation	Area error was 4-7 % (larger for objects with long perimeters such as stars).	The permissive threshold dilated masks. Per-object half-maximum edge refinement removed the bias; median error fell to 0.64 %. An attempted 'half-perimeter' correction was tested and rejected because it over-corrected.

Challenge	Symptom	Resolution
Watershed-cut circles misclassified	Circles split by watershed have a flat side; confidence fell below the threshold.	Training data now include circles separated by the real watershed step.
Digital-contour artefacts	A perfect circle has circularity about 0.90, not 1.0, because pixel contours overestimate perimeter.	Features are learned from real pipeline output, so the classifier sees the same distribution; unit-test expectations were corrected.
Light and dark backgrounds	Grey-level Otsu needs a polarity guess and loses low-contrast colours.	Switched to Lab-distance-from-border segmentation.
Evaluating without a labelled dataset	No annotated real data were available.	Built a synthetic generator with exact ground truth; honestly noted that real-photo performance still has to be verified.

A further lesson was that very high scores on synthetic data can hide weaknesses: the noise sweep and the baseline comparison were added specifically to look for failure modes, and the report states plainly where the evidence stops.

12. Learnings and Key Takeaways

- **Colour space choice matters:** measuring distance in CIE-Lab made segmentation independent of object colour, something grey-level thresholding cannot do.
- **Measure, do not guess:** the area bias was invisible by eye and was found only by comparing against exact ground truth; a quantitative evaluation loop made each fix verifiable.
- **Classical CV is still powerful:** with a few well-chosen descriptors a small SVM is perfect on this task - and fully explainable, unlike a black-box network.
- **Train on what you will test on:** features computed from the true pipeline output prevented train/inference mismatch.
- **Edge cases dominate engineering effort:** touching objects, tiny noise cores and boundary bias took more time than the happy path.
- **Software engineering practice pays off:** modular design, custom exceptions, logging and 70 tests made refactoring safe (several algorithm changes were validated in seconds).
- **Be honest about limits:** synthetic results are an upper bound; documenting this is part of a trustworthy evaluation.

13. Future Enhancements

- Validate and fine-tune on a **real photo dataset** (shadows, glare, textured surfaces) and add data augmentation.
- Add **perspective correction** with a four-point homography or a checkerboard / ArUco marker, plus lens-distortion calibration.
- Support **arbitrary object categories** with a CNN (e.g. YOLO or Mask R-CNN) while keeping the classical measurement stage.
- Web or desktop **GUI** and live camera mode for real-time counting.
- Handle **partial occlusion and touching non-circular objects** with learned instance segmentation.

- Automatic detection of the reference marker (colour-independent) and sub-pixel edge fitting for finer measurement.
- Packaging as a pip-installable library and a continuous-integration workflow running the test suite on every push.

14. References

[1] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.

[2] OpenCV documentation - Image Segmentation with Watershed Algorithm, Contour Features, Morphological Transformations. https://docs.opencv.org

[3] Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), 62-66.

[4] Vincent, L., and Soille, P. (1991). Watersheds in digital spaces: an efficient algorithm based on immersion simulations. *IEEE TPAMI*, 13(6), 583-598.

[5] Hu, M.-K. (1962). Visual pattern recognition by moment invariants. *IRE Transactions on Information Theory*, 8(2), 179-187.

[6] Zuiderveld, K. (1994). Contrast Limited Adaptive Histogram Equalization. In *Graphics Gems IV*, Academic Press, 474-485.

[7] Cortes, C., and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297.

[8] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

[9] Szeliski, R. (2022). *Computer Vision: Algorithms and Applications*, 2nd ed. Springer.

[10] Gonzalez, R. C., and Woods, R. E. (2018). *Digital Image Processing*, 4th ed. Pearson.

[11] Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357-362.

[12] VITyarthi - Build Your Own Project: General Project Instructions and Submission Guidelines (course document).